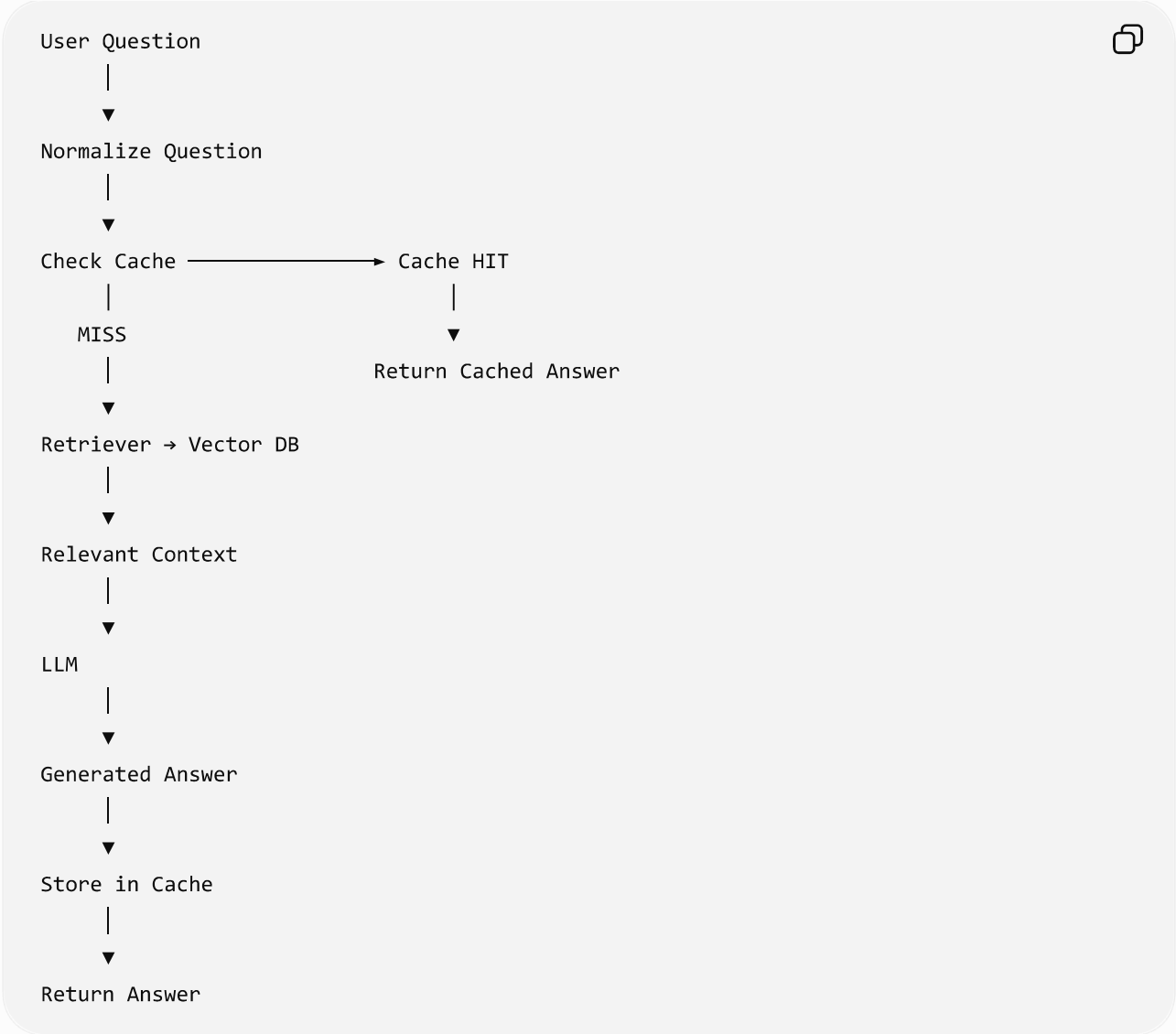
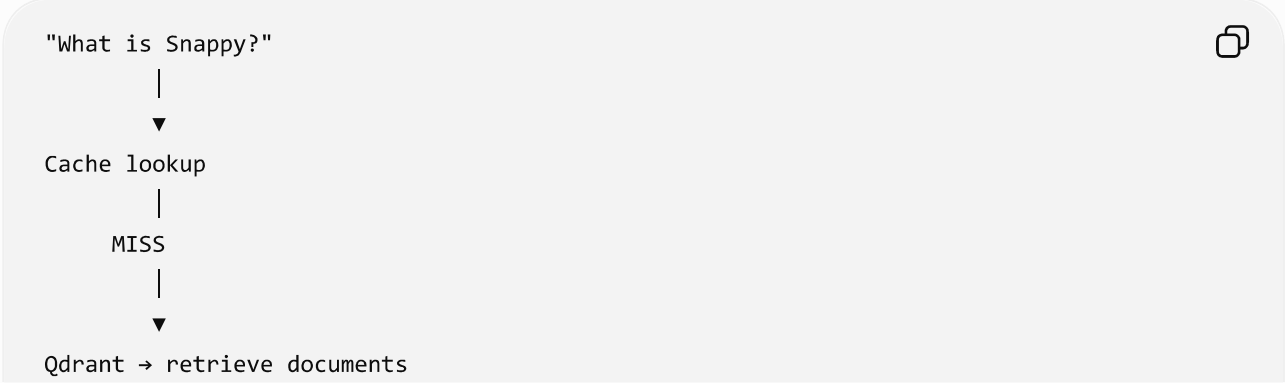
This is essentially describing **LLM/RAG response caching** and **prompt caching**, which solve related but different problems.

1. RAG response caching

The flow is:



For example:



```
        |
        ▼
LLM → "Snappy is..."
        |
        ▼
Redis:
{
  key: "website:snappy:v1:what-is-snappy",
  answer: "Snappy is..."
}
```

Next time:

```
"What is Snappy?"
        |
        ▼
Redis
        |
      HIT
        |
        ▼
"Snappy is..."
```

So you avoid:

embedding/retrieval → prompt construction → LLM generation

and get a much faster response.

---

2. The important distinction: RAG cache vs Prompt Cache

These are often confused.

Feature	RAG Response Cache	Prompt Caching
What is cached?	Final/generated answer	Reusable input tokens
Cache location	Redis/SQLite/etc.	LLM provider infrastructure
LLM called on HIT?	Usually **No**	**Yes**
Retrieval required?	No	Depends
Reduces latency	✅ Significant	✅
Reduces LLM cost	✅	✅ Significant

Feature	RAG Response Cache	Prompt Caching
Handles slightly different questions	Usually no	✅
Best for	Repeated questions	Large repeated context/system prompts

RAG response cache

```
Question
   ↓
Cache
   ↓ HIT
Final Answer
```

Anthropic-style prompt caching

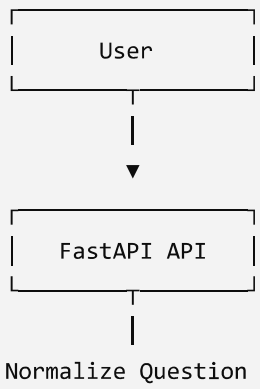
```
Large Document / System Prompt
              ↓
         Cached Prefix
              ↓
User Question → LLM
              ↓
           Answer
```

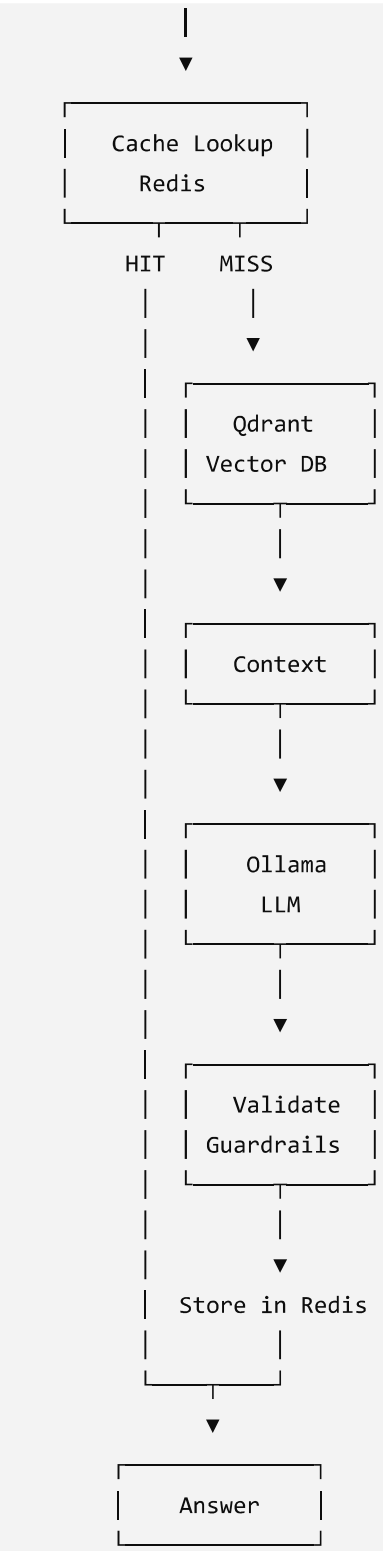
The model still runs for every question.

---

3. For your Advanced RAG architecture

Since you're working with **FastAPI** + **Qdrant** + **Ollama**, I'd recommend a more production-oriented design than simply caching the question.





4. But there's an even better approach: Semantic Cache

Exact caching has a limitation.

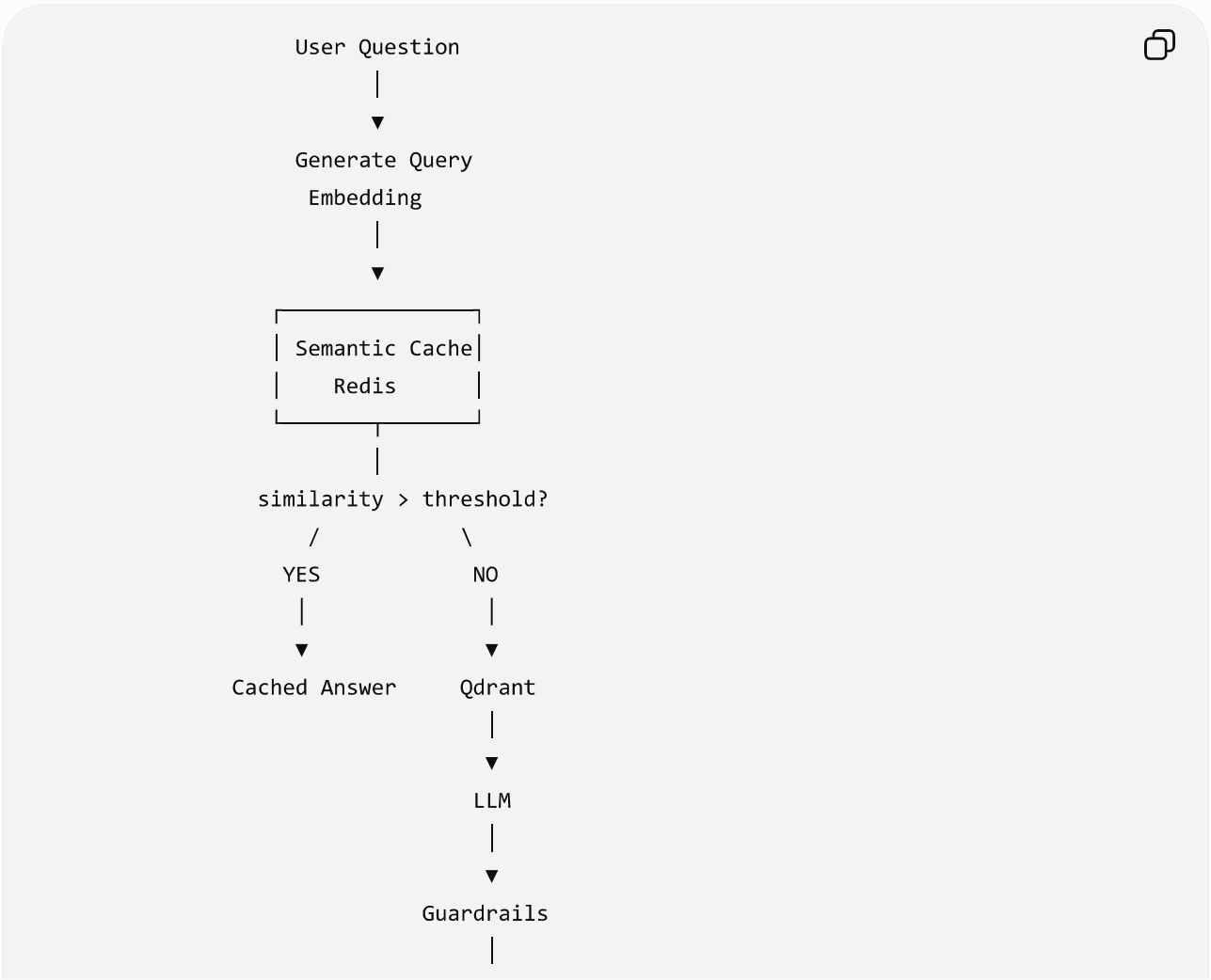
These two questions are different strings:

```
"What is Snappy?"
"What does Snappy do?"
```

An exact cache treats them as different.

A **semantic cache** can determine that they are asking essentially the same thing.

So your architecture can become:



```
                ▼
          Semantic Cache
                │
                ▼
             Answer
```

This is particularly useful for your RAG system.

---

5. Cache invalidation is the critical part

The lecture correctly highlights this.

Imagine your documentation changes:

```
Documentation v1
     ↓
"What is Product X?"
     ↓
Cached answer:
"Product X supports A, B and C"
```

Then documentation changes:

```
Documentation v2
     ↓
Product X no longer supports C
```

But Redis still contains:

```
"What is Product X?"
        ↓
"A, B and C" ❌
```

That's dangerous for RAG.

A good solution is **versioned cache keys**:

```
rag:v1:website123:question_hash
```

After updating documents:

```
rag:v2:website123:question_hash
```

Now old answers are automatically bypassed.

You can also use:

```
TTL
+
Document Version
+
Model Version
+
Prompt Version
```

For example:

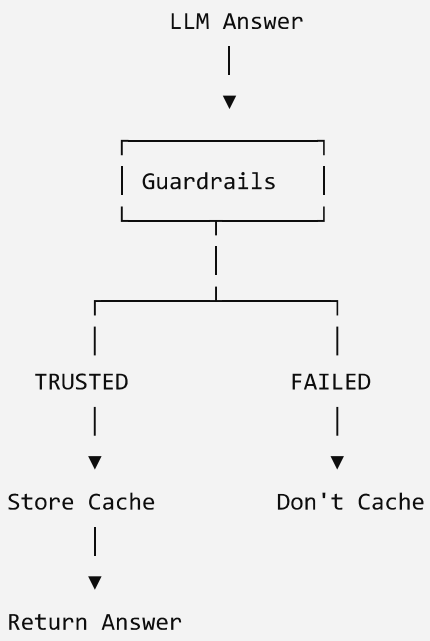
```
cache_key =
  hash(
    tenant_id +
    document_version +
    prompt_version +
    model_version +
    normalized_question
  )
```

---

6. Don't cache every answer blindly

This is especially important for your **RAG guardrails** architecture.

A safer pipeline is:



You can apply conditions such as:

```
faithfulness > threshold
context_precision > threshold
answer_relevancy > threshold
injection_detected == false
blocked_reason == null
```

Only then:

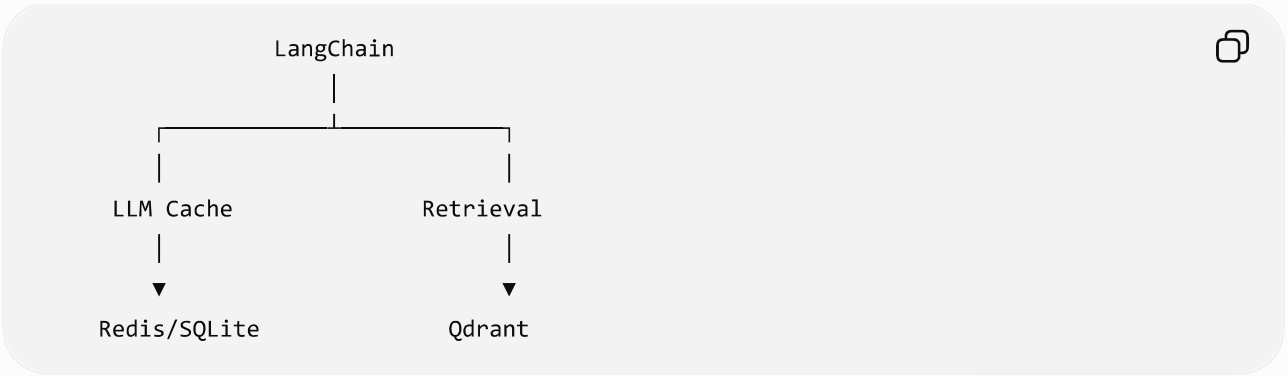
```
Redis SET cache_key answer
```

This prevents a hallucinated answer from becoming a **permanent cached hallucination**.

---

7. Where LangChain fits

Conceptually:



LLM caching primarily avoids repeated LLM calls for identical inputs.

But for your production RAG system, I'd separate:

```
L1 → Exact Response Cache
L2 → Semantic Cache
L3 → Retrieval Cache
L4 → LLM/Prompt Cache
```

For example:

```
User
 │
 ▼
L1 Exact Cache ——— HIT ——► Answer
 │ MISS
 ▼
L2 Semantic Cache —— HIT ——► Answer
 │ MISS
```

```
 ▼
L3 Retrieval Cache
 │
 ▼
Qdrant
 │
 ▼
L4 Prompt/LLM Cache
 │
 ▼
Ollama / LLM
```

That gives you a much stronger **production-grade RAG caching architecture** than simply putting `InMemoryCache` in front of the LLM.

The key takeaway is:

> **Response caching caches the answer; semantic caching caches the meaning; prompt caching caches reusable input context.**

For your RAG platform, **semantic response caching + cache versioning + guardrail validation + Redis** would be the most interesting implementation to demonstrate in an architecture/interview project.